The Spectrum of Conformance Checking

- Conformance checking is about detecting and analyzing deviations between observed/recorded and modelled behavior.

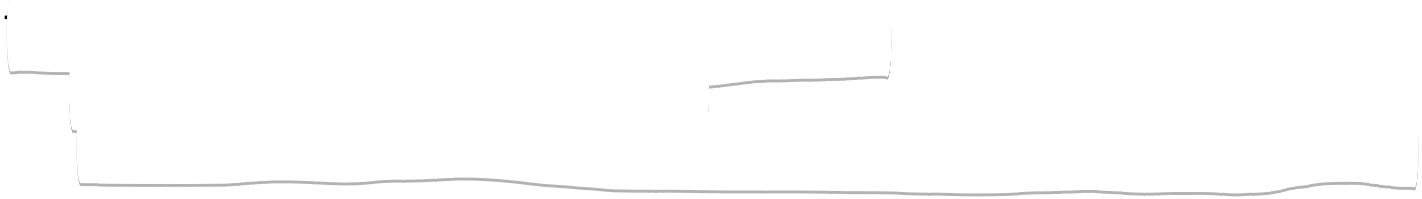
Idea of Rule Checking

General idea:

- Process model constrains the possible behaviour of a process
- Formalise constraints by rules that execution sequences have to obey to
- Then, verify these rules for the traces of a log

Rule checking focuses on the fitness dimension

- Traces are fitting, if they satisfy the rules
- Traces are non-fitting, if rules are violated



What rules?

Often: unary and binary rules over activities

- Specify how a single activity is executed or how the executions of a pair of activities relate to each other
- N-ary rules would lead to combinatorial explosion

Common types of rules:

- Cardinality rules: Upper and/or lower bound for the number of executions of an activity
- Precedence and response rules: An activity is always preceded/succeeded by another one
- Ordering rules: Activities are independent, but if they are both executed, they are ordered
- Exclusiveness rules: Activities must not be executed in the same case

Feedback on Non-Conformance

Rule-checking gives information on fitness of trace

- Is the model well-suited to explain the recorded trace?
- Yes, if rules are satisfied; no, if rules are violated
- Fine-granular feedback on the level of activities

Aggregated feedback on the level of a log

- Rules that are frequently violated point to general conformance issues

Fitness Measure

Counting of satisfied rules enables quantification of fitness:

$$\text{fitness}(\sigma, M) = \frac{|\{r \in R_M \mid r \text{ is satisfied by } \sigma\}|}{|R_M|}$$

$$\text{fitness}(L, M) = \frac{|\{r \in R_M \mid r \text{ is satisfied by all } \sigma \in L\}|}{|R_M|}$$

Obviously, the above measures depend on the chosen set of rules

Expressiveness of Rules Cont.

Can this be fixed by introducing further rules?

In theory, yes, BUT:

- An exponential number of rules would be needed to fully characterise the behaviour of process model (specifically, a regular language)

Hence, if striving for completeness, rule checking becomes

- Inefficient—an exponential number of rules need to be checked
- Ineffective—the result of rule checking is overwhelming to users due to its exponential size

Idea of Token Replay

Idea

- Try to “replay” each trace in the model by executing activities according to the trace
- Not ok:
  - Activity is executed although it is not enabled
  - Tokens remain in the model and are not consumed

Procedure:

1) Initialise the replay procedure by setting the *first event* of the trace as the *current event* of the replay
2) Locate the activity for which execution is signalled by the *current event* of the trace in the model
3) Replay the *current event* of the trace by executing the identified activity in the *current state* of the model by:
   - Consuming a token on the incoming arc of the activity, even if there is none (if so, record as missing)
   - Producing a token on the outgoing arc of the respective task (if there is any)
4) Set the state reached as the *current state* of the process model
5) If the current event is not the last in the trace, consider the next event in the trace as the current event and continue the replay from step 2 onwards

Fitness Measure

Determine ratios based on missing and remaining tokens:

$$\text{fitness}(\sigma, M) = \frac{1}{2}\left(1 - \frac{missing(\sigma, M)}{consumed(\sigma, M)}\right) + \frac{1}{2}\left(1 - \frac{remaining(\sigma, M)}{produced(\sigma, M)}\right)$$

$$\text{fitness}(L, M) = \frac{1}{2}\left(1 - \frac{\sum_{\sigma \in L} missing(\sigma, M)}{\sum_{\sigma \in L} consumed(\sigma, M)}\right) + \frac{1}{2}\left(1 - \frac{\sum_{\sigma \in L} remaining(\sigma, M)}{\sum_{\sigma \in L} produced(\sigma, M)}\right)$$

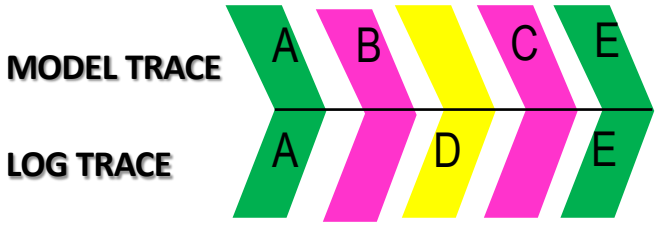
Limitations

Replay is fast and works reasonable well in practice

But: Replay may become non-deterministic in two situations:

- It may be impossible to unambiguously locate the activity to execute when replaying an event
- It may be impossible to unambiguously characterize the state reached after replaying an event

Conformance Artefact: ~~Alignments~~



SYNCHRONOUS MOVE: LOG AND MODEL AGREE

MODEL MOVE: TASKS NOT RECORDED THAT SHOULD BE EXECUTED

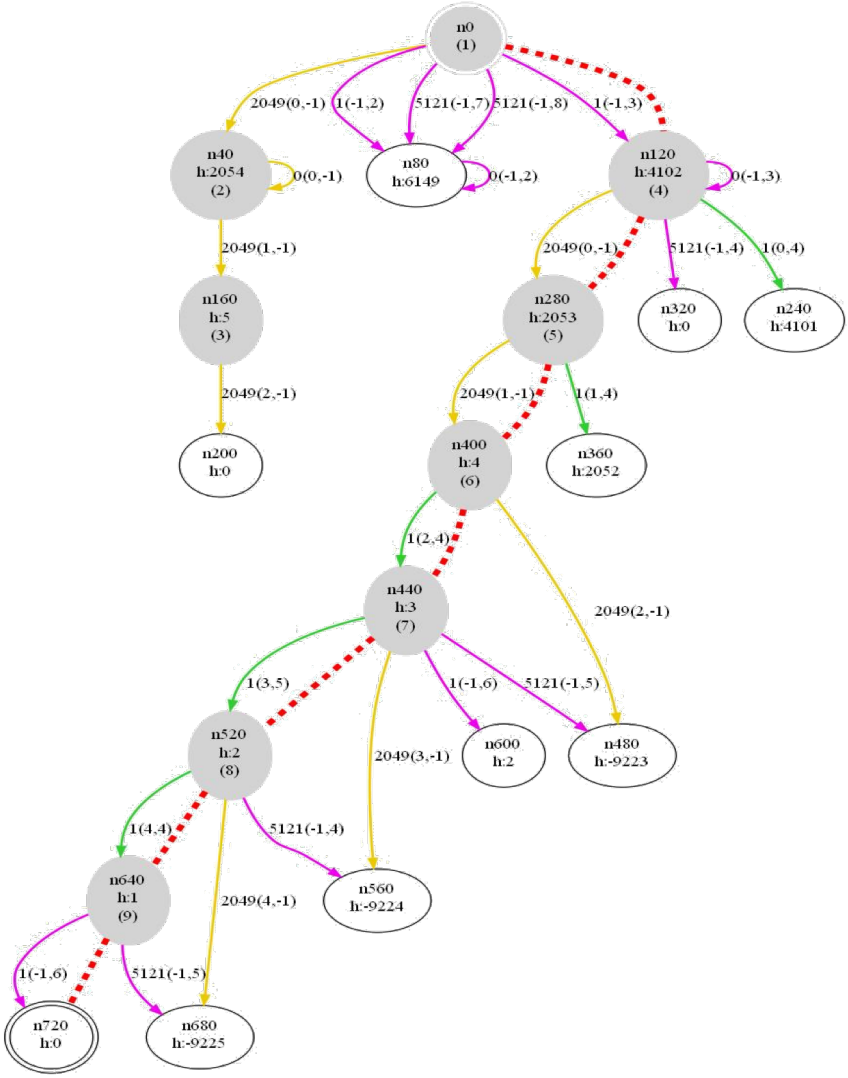
LOG MOVE: EVENTS RECORDED THAT CANNOT BE EXECUTED

Computing Alignments through A*

- Reference technique for computing alignments
- Can guarantee optimality for an arbitrary cost function on deviations
- Can be adjusted to find not just one but all optimal alignments

Computing Alignments

- Each node is a combination of a state in the model and the remaining events in the trace
- Each arc is a move on model, move on log or a synchronous move
- A heuristic function estimates the remaining distance to the final node (i.e. model in a final state and all events executed)
- We find a shortest path!



Dijkstra with heuristic instead of queue

Computing Alignments (A* with fast lowerbound)

```
Initialize HashBackedPriorityQueue q
While (peek(q) is not target t)
  VisitedNode n = head(q)
  If the estimate for node(n) is not exact
    Compute the exact estimate for the remaining distance to t and
    add n to the priority queue
    continue
  add n to the considered nodes
  For each edge in the graph from node(n) to m
    If m was considered before, continue
    If m is in the queue with lower cost, continue
    If m is in the queue with higher cost, update and reposition it
    If m is new,
      compute a fast lowerbound for the remaining distance to t
      v = new VisitedNode(m)
      set n a predecessor for v
      add v to the priority queue
Return head(q)
```

O(log(size q)+α)

BIG PROBLEM

?

O(log(size q)+ α)

O(log(size q)+ α)

O(1)

O(log(size q)+ α)

Estimating remaining distance I

- Naïve estimation: 0 Trivial
- LP-based estimation: "Polynomial"
  - Minimize **c.x**
    Where A.**x** = **r**
    c.x ≥ c.x′
- ILP-based estimation: Exponential
- Hybrid ILP (1 sec for "I" part) Exponential
- Caching LP basis solutions Exponential

Little to no difference in practice

Parikh Vector

The Parikh vector as a basic concept of sequences:

- Given a sequence of elements of some alphabet Σ, the Parikh vector of a sequence $\sigma \in \Sigma^*$ over this alphabet is a function $\overrightarrow{\cdot} : \Sigma^* \to \mathbb{N}$ resulting in a vector of the cardinalities of all elements in σ
- An implicitly assumed order of elements in Σ determines the order in the vector

Example in our context with alphabet $\Sigma = \{A, B, C, D\}$:

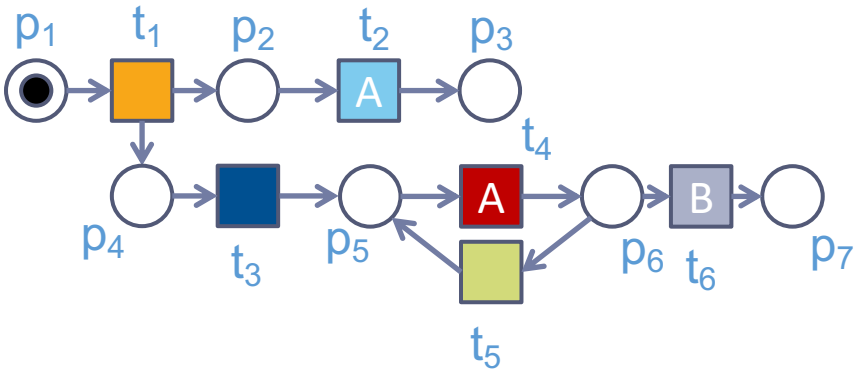
- Trace $\sigma = \langle A, A, B, A \rangle : \vec{\sigma} = [3, 1, 0, 0]^T$

Incidence Matrix

Represent the structure of a Petri net (P,T,F) in terms of the flow relation as a matrix

- Matrix of |P| rows and |T| columns
- Values of −1 and 1 indicate the presence of a flow arc from a place to a transition, or from a transition to a place, respectively

Example:



	t_1	t_2	t_3	t_4	t_5	t_6
p_1	−1					
p_2	1	−1				
p_3		1				
p_4	1		−1			
p_5			1	−1	1	
p_6				1	−1	−1
p_7						1

Marking Equation Cont.

Considering all places of a Petri net system, we get the Marking Equation:

$$\overrightarrow{\mathrm{M}} = \overrightarrow{\mathrm{M}_i} + C \cdot \vec{\sigma}$$

where $\overrightarrow{\mathrm{M}}$ and $\overrightarrow{\mathrm{M}_i}$ are place vectors of the respective markings and C is the incidence matrix.

Markings are non-negative, so that:

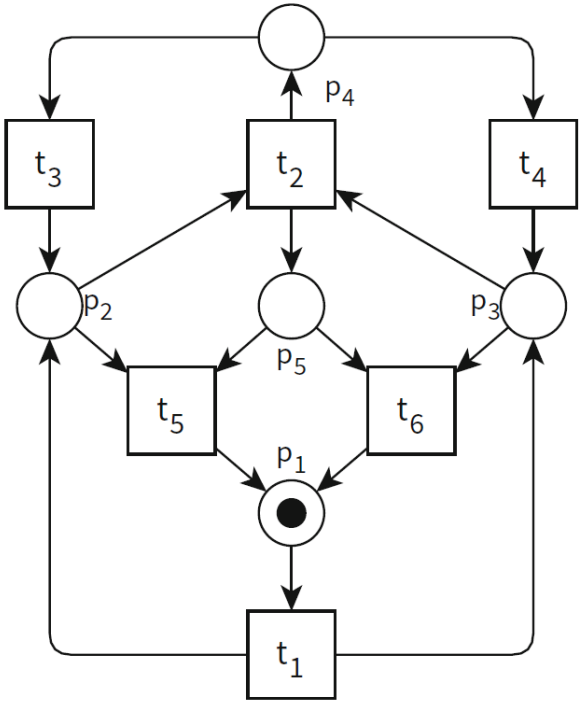
$$\overrightarrow{\mathrm{M}} = \overrightarrow{\mathrm{M}_i} + C \cdot \vec{\sigma} \geq \vec{0}$$

The above yields a necessary, but not sufficient conditions for reachable markings

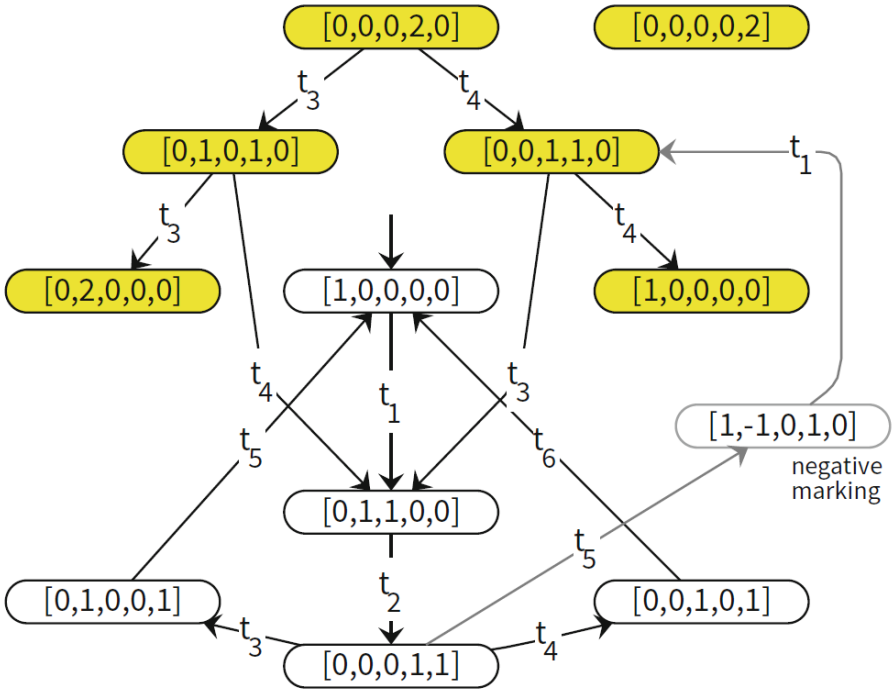
- All reachable markings satisfy the above inequality
- But, finding a marking that satisfies it, does not mean that the marking is reachable

All markings satisfying the inequality are *potentially reachable*

Marking Equation Example



Potentially reachable markings
(highlighted are not reachable):



Consider a firing sequence:

$\langle t_1, t_2, t_5, t_1, t_3 \rangle$

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -1 & 0 & 0 & 0 & 1 & 1 \\ 1 & -1 & 1 & 0 & -1 & 0 \\ 1 & -1 & 0 & 1 & 0 & -1 \\ 0 & 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Another Heuristic for A*

Idea:

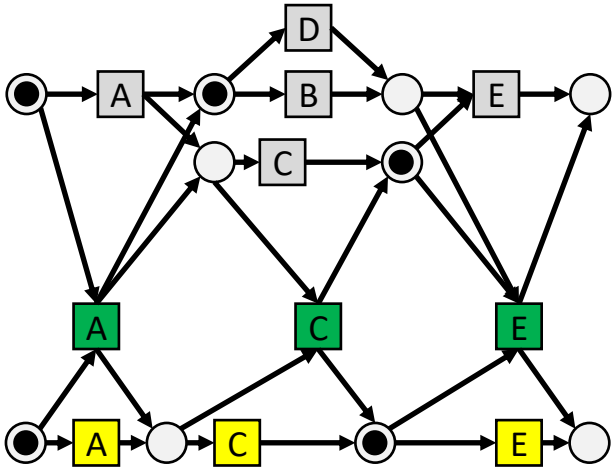
- Sequence of transition firings in the product state space yields an integer solution to the marking equation
- Cost function over this sequence is the alignment cost

With M as some marking and $\mathrm{M_f}$ as the final marking of the product state space, the heuristic is defined as ($\delta(\vec{x})$ being the aggregated cost of the non-zero transitions in $\vec{x}$) :

$$h(M)\begin{cases}\infty & \text{if there is no solution to } \overrightarrow{M} + C \cdot \vec{x} = \overrightarrow{M_f} \\ \delta(\vec{x}) & \text{where } \vec{x} \text{ is a solution to } \overrightarrow{M} + C \cdot \vec{x} = \overrightarrow{M_f} \text{ and } \vec{x} \geq \vec{0} \\ & \text{with } \delta(\vec{x}) \text{ being mininal}\end{cases}$$

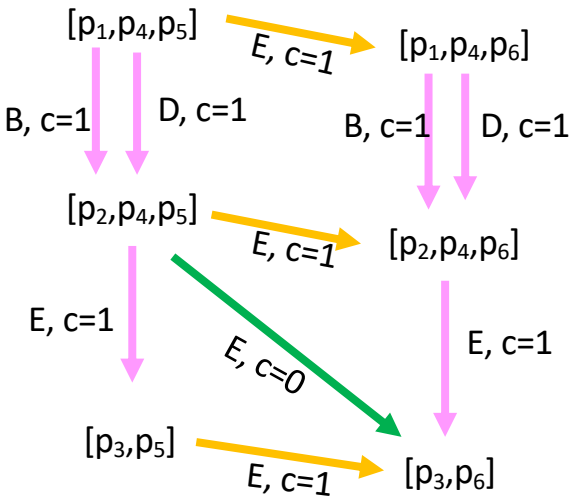
Estimating remaining distance II

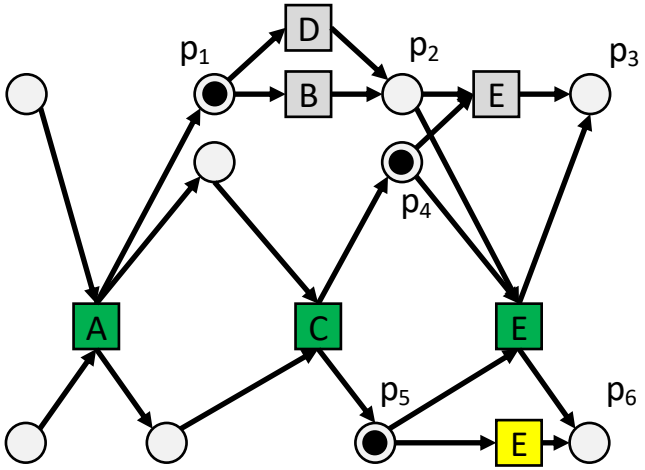
- Consider the model on the right
- And the trace <A,C,E>
- Make the synchronous product
- We reach the marking after A and C



Estimating remaining distance III

What is the remaining cost of getting from $[p_1,p_4,p_5]$ to $[p_3, p_6]$

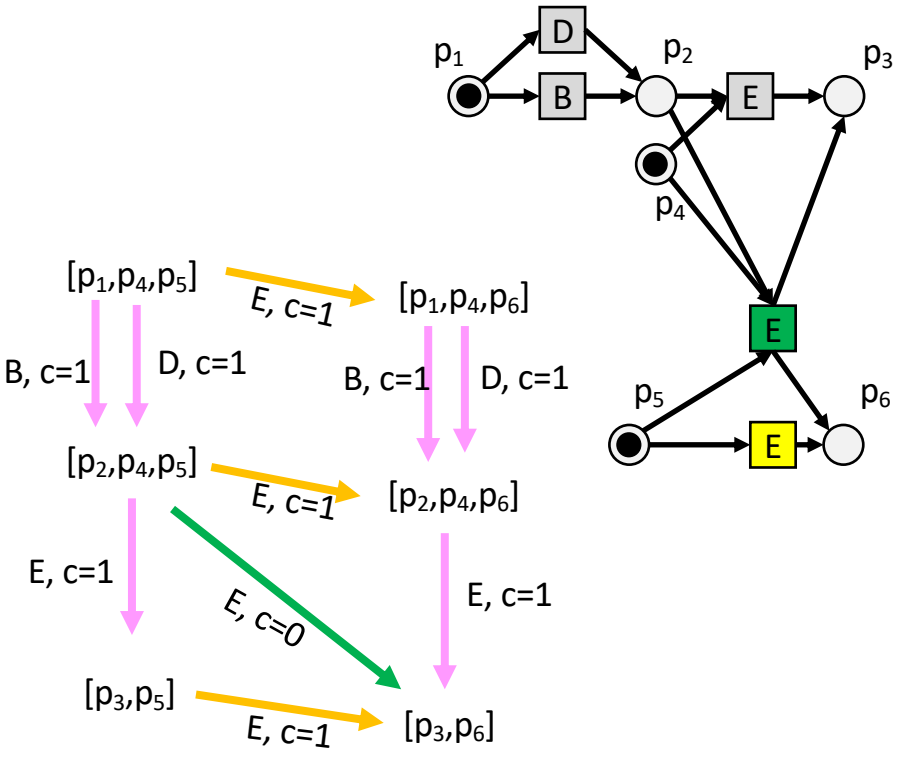




Estimating remaining distance IV

Estimators:

- Dijkstra: 0
- State-equation: 1
  solution: {(B,>>)(E,E)}
- State-equation: 1
  solution: {$^1/_2$(B,>>), $^1/_2$(D,>>),(E,E)}



Implementations: Estimator Versions

Petri nets:

- Dijkstra (estimator 0)
- Naive (estimator parikh)
- ILP (estimator using ILP)
- Basis caching
- Fast lowerbounds

Process Trees:

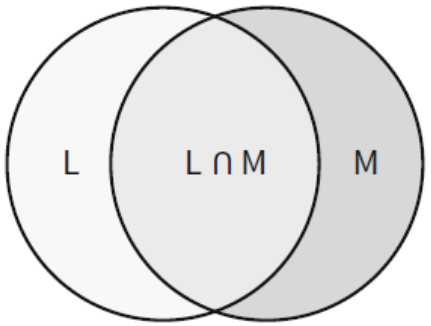
- Naive (estimator parikh)
- LP (estimator using LP)
- Hybrid ILP (ILP & LP)
- Special constraints for OR
- Fast lowerbounds
- Basis caching within LP
- Statespace reduction (stubborn sets)

Alignment-based Measures

Alignments enable the quantification of the relation between an event log and a process model

In the context of conformance checking:
Fitness between a log and a model is most relevant

- Assume that the model is normative
- Quantify the extent of deviations of a trace from a model



Fitness Measures

Absolute fitness of a trace σ regarding a given model

- Defined by the cost δ of the optimal alignment of σ with the model
- Using the standard cost function, this corresponds to the number of moves in log and moves in model

Absolute fitness of a multiset $S = [\sigma_1^{n_1}, \dots, \sigma_m^{n_m}]$ of traces regarding model M with complete execution sequences Π

- Sum of costs of optimal alignments of traces

$$\mathrm{fcost}(S, \Pi) = \sum_{1 \le i \le m} \mathrm{n_i}\ \delta(\gamma^*(\sigma_i, \Pi))$$

Normalised Fitness

Various solutions to normalise the absolute fitness measure

Here: Division by maximal possible value

- Assume that a move in both (x, y) happens only if $x = y$
- Worst case: alignment is built only from moves in model and moves in log
- With S as the multiset of traces, the cost of moving through the log without moving in the model is:

$$\mathrm{c}(\mathrm{S}) = \sum_{1 \le i \le m} \mathrm{n_i} \sum_{x \in \sigma_i} \delta(x, \bot)$$

- The cost of moving only in the model (via the least expensive path) is:

$$\mathrm{c}(\Pi) = \min_{\pi \in \Pi} \sum_{y \in \pi} \delta(\bot, y)$$

Normalised fitness measure:

$$\mathrm{fitness}(\mathrm{S}, \Pi) = 1 - \frac{\mathrm{fcost}(\mathrm{S}, \Pi)}{c(\mathrm{S}) + \sum_{1 \le i \le m} \mathrm{n_i}\, \mathrm{c}(\Pi)}$$

Assess Precision

Quantify the behaviour of the model not seen in the log

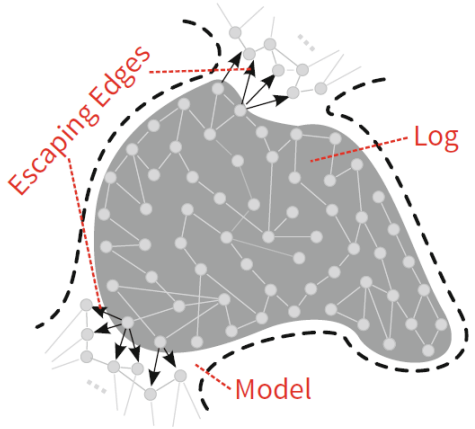
Assumption: All traces fit the model

Assumption: Model is deterministic

- In any state, there cannot be two transitions with the same label

General idea

- Based on traces of the log, characterise states in the model with "escaping edges"
- Those are transitions representing behaviour that immediately follows behaviour of the log but is not contained in the log



Precision Measure

Determine the language of "escaping edges", denoted L_{esc}

- It contains all sequences of activity labels $\sigma \cdot \langle a \rangle$, such that
  - σ is a prefix of a trace in the log and a prefix of an execution sequence of the model
  - $\sigma \cdot \langle a \rangle$ is not a prefix of a trace in the log, but still a prefix of an execution sequence of the model

Precision of a log as a set of traces $L = [\sigma_1, \ldots, \sigma_m]$ and a model M with execution sequences Π:

$$\text{precision} = \frac{|\text{Prefix}(L \cap \Pi)|}{|\text{Prefix}(L \cap \Pi)| + |L_{esc}|}$$

where Prefix(X) is the set of all prefixes of language X.

Alignment-based Precision Measure

Alignments enable us to drop the assumption of fitting traces in the computation of precision

- Intuitively, the alignment gives an execution sequence best resembling an unfitting trace
- This execution sequence is used when building the joint language
- Instead of relying on $L \cap \Pi$, we rely on $L_{align} = \{ \gamma^*(\sigma, \Pi)_{|2} \mid \sigma \in L \}$ where $\gamma^*_{|2}$ is the projection of the alignment on the second component (i.e., the execution sequence)

Then, redefine the language of "escaping edges", denoted L_{esc} based on alignments:

- It contains all sequences of activity labels $\sigma \cdot \langle a \rangle$, such that
  - σ is a prefix of an element of L_{align}
  - $\sigma \cdot \langle a \rangle$ is not a prefix of a trace in the log, but still an element of L_{align}

Precision is then measured as :

$$\text{precision} = \frac{|\text{Prefix}(L_{align})|}{|\text{Prefix}(L_{align})| + |L_{esc}|}$$